

Stoquify Future-Ready Copilot Backbone Architecture Prompt

Document type: Execution-ready architecture and product strategy prompt Prepared for: Stoquify leadership, product, engineering, security, operations, and commercial teams Date: 15 July 2026 Status: Ready to execute

Professional Prompt

Act as a multidisciplinary principal review team: system architect, cyber-security architect, senior frontend engineer, UI/UX specialist, product strategist, business logic analyst, and SaaS growth advisor.

Act additionally as an enterprise AI-platform architect, multi-agent systems engineer, knowledge-systems architect, FinOps specialist, reliability engineer, and OHADA/SYSCOHADA-aware business-platform strategist.

Mission

Architect a magnificent, enterprise-grade Copilot backbone and operational spine for Stoquify that is valuable today, extensible for the future, and capable of accommodating both the existing agents and skills and those introduced later.

The objective is not to add a superficial chatbot or a collection of disconnected agents. Design a coherent, governed, modular intelligence and execution platform that can become Stoquify's trusted operating layer-helping users understand what matters, decide what to do, complete authorized work, and preserve verifiable evidence of every material outcome.

The architecture must allow agents, skills, models, tools, workflows, integrations, country packs, knowledge sources, and user experiences to evolve independently without fragmenting the platform.

It must operate smoothly, safely, efficiently, and economically while increasing Stoquify's:

- daily and hourly usefulness;
- workflow completion rate;
- user trust;
- module adoption;
- customer retention;
- expansion revenue;
- institutional intelligence;
- automation capability;
- ecosystem potential; and
- long-term competitive defensibility.

Primary Objective

Produce a complete target architecture and phased implementation blueprint for a future-ready Stoquify Copilot platform that:

- 1 Gives all current agents and skills a consistent place in the architecture.
- 2 Allows future agents and skills to be added, tested, versioned, governed, deployed, monitored, and retired without destabilizing the system.

- 3 Connects intelligence to real Stoquify workflows rather than generating isolated conversational answers.
- 4 Preserves deterministic, service-owned business truth.
- 5 Enforces tenant isolation, RBAC, module entitlements, privacy, approval, audit, evidence, and financial controls.
- 6 Supports advisory, assistive, supervised, and eventually policy-bounded autonomous operation.
- 7 Creates measurable short-term customer value while building durable long-term moats.
- 8 Remains model-agnostic, provider-portable, observable, cost-controlled, resilient, and scalable.

Required Discovery Before Designing

Begin with evidence-first discovery. Do not propose the architecture from assumptions alone.

Inspect the Existing Platform

Review the relevant parts of:

- app/
- actions/
- services/
- components/
- hooks/
- lib/
- config/
- prisma/
- scripts/
- tests and release-verification infrastructure;
- existing agent and skill definitions;
- existing Copilot, AI, workflow, audit, RBAC, entitlement, notification, reconciliation, and observability code; and
- relevant reports under docs/, what-next/, and innovation/.

Use Existing Architectural Evidence

Check graphify-out/ and the available graph reports before making dependency, routing, component, service-boundary, or impact claims:

- graph_components.json
- graph_actions.json
- graph_app.json
- graph_hooks.json
- graph_types.json
- corresponding GRAPH_REPORT_*.md files

Inventory the Existing Intelligence Assets

Create an evidence-backed inventory of:

- existing agents;

- existing skills;
- prompts and system instructions;
- tools and callable actions;
- workflow engines;
- model providers;
- retrieval or knowledge components;
- country packs;
- evaluation mechanisms;
- audit and evidence systems;
- human-approval workflows;
- notification and escalation mechanisms;
- existing telemetry and observability;
- duplicated or overlapping capabilities; and
- architectural gaps.

For every existing agent and skill, determine:

- its purpose;
- target user or role;
- domain ownership;
- required inputs;
- generated outputs;
- tools and permissions;
- risk classification;
- evidence requirements;
- approval requirements;
- dependencies;
- current maturity;
- duplication or overlap; and
- where it belongs in the proposed architecture.

Do not invent repository capabilities that cannot be verified. Clearly distinguish confirmed current-state findings from recommendations.

Required Architectural Design

Design the Copilot as a layered enterprise platform. At minimum, evaluate and define the following architectural planes.

1. Experience and Interaction Plane

Define how users interact with the Copilot across:

- global Copilot interface;
- contextual assistance inside modules;

- role-specific command centers;
- daily operating briefs;
- exception queues;
- guided workflows;
- notifications and escalations;
- mobile and low-bandwidth environments;
- multilingual interaction;
- explainability and evidence inspection; and
- accessibility.

The Copilot must feel embedded in the work, not isolated in a chat window.

Define how conversations, cases, tasks, recommendations, approvals, and completed outcomes remain continuous across surfaces.

2. Agent Orchestration and Control Plane

Design the component responsible for:

- intent classification;
- agent selection;
- skill selection;
- workflow planning;
- task decomposition;
- context assembly;
- tool routing;
- dependency management;
- concurrency;
- retries and timeouts;
- cancellation;
- checkpointing;
- compensation and rollback;
- human escalation; and
- final response composition.

Determine whether Stoquify needs a central orchestrator, domain supervisors, specialized agents, event-driven agents, deterministic workflow engines, or a hybrid architecture.

Avoid an uncontrolled "agent swarm." Every agent must have a bounded responsibility, defined inputs and outputs, permitted tools, and measurable success conditions.

3. Agent and Skill Registry

Design a first-class registry for agents and skills. Each registered capability should have a machine-readable contract containing:

- unique identifier;
- name and purpose;

- owner;
- version;
- lifecycle status;
- supported domains and countries;
- eligible roles and module entitlements;
- required inputs and output schema;
- permitted tools and permission scope;
- risk class and required approvals;
- evidence policy;
- supported languages;
- cost budget and latency target;
- model compatibility;
- evaluation suite;
- release requirements;
- rollback procedure; and
- deprecation policy.

Define how capabilities are discovered, composed, versioned, certified, deployed, monitored, and retired.

4. Tool and Action Gateway

Design a secure gateway between agents and Stoquify's real services.

Agents must not write directly to databases or calculate authoritative financial, payroll, stock, or accounting truth independently.

The gateway must enforce:

- tenant isolation;
- authenticated identity;
- RBAC;
- module entitlement;
- organization and location scope;
- fresh authentication where required;
- purpose limitation;
- input and output validation;
- idempotency;
- transaction boundaries;
- rate limits;
- approval policies;
- maker-checker controls;
- sensitive-data redaction;
- audit events; and

- safe error handling.

Classify tools as read-only, analytical, draft-producing, reversible write, high-impact write, financial or statutory action, and prohibited action. Define the control requirements for every class.

5. Context, Knowledge, and Memory Plane

Architect the different forms of memory without mixing their responsibilities:

- current conversation context;
- active workflow or case state;
- tenant operational context;
- user preferences;
- organization policies;
- product documentation;
- approved business procedures;
- country-pack knowledge;
- regulatory and statutory sources;
- prior decisions and evidence; and
- aggregated outcome learning.

Define source ownership, retrieval policies, freshness, provenance, confidence, retention, encryption, redaction, consent, tenant boundaries, deletion, correction, and exportability.

The architecture must prevent cross-tenant leakage and uncontrolled long-term memory.

6. Event and Workflow Spine

Design an event-driven operational spine through which important Stoquify events can trigger governed assistance.

Consider events such as:

- sale completed;
- payment received;
- settlement delayed;
- reconciliation mismatch detected;
- stock threshold reached;
- abnormal adjustment recorded;
- supplier invoice received;
- approval delayed;
- payroll input incomplete;
- statutory deadline approaching;
- close control failed; or
- evidence expired.

Define canonical business events, event schemas, producers and consumers, idempotency, ordering, replay, dead-letter handling, correlation identifiers, case creation, workflow state, SLA tracking, escalation, and resolution evidence.

Show how a single event can coordinate multiple roles and modules without duplicating business truth.

7. Trust, Security, and Governance Plane

Design governance as part of the core architecture. Include:

- identity and authorization;
- least-privilege tool access;
- prompt-injection defenses;
- data-loss prevention;
- model-provider isolation;
- secrets management;
- tenant isolation;
- approval policies;
- evidence requirements;
- immutable audit trails;
- redaction;
- policy enforcement;
- abuse prevention;
- incident response;
- kill switches;
- capability revocation; and
- regulatory review.

Define an autonomy ladder:

- 1 Explain.
- 2 Recommend.
- 3 Prepare a draft.
- 4 Execute with confirmation.
- 5 Execute under maker-checker approval.
- 6 Execute automatically within an explicit policy.
- 7 Escalate when policy, confidence, or authority is insufficient.

Specify the release gates required before a capability can move to a higher level.

8. Evaluation and Assurance Plane

Design evaluation at the workflow level, not merely at the language-model level. Include:

- offline evaluation datasets;
- golden cases;
- adversarial and security tests;
- tenant-isolation tests;
- country-pack correctness tests;
- hallucination and unsupported-claim checks;
- evidence-completeness checks;

- tool-selection accuracy;
- workflow-completion accuracy;
- human correction rate;
- regression tests;
- shadow deployment;
- canary release;
- rollback; and
- post-release monitoring.

Define clear standards for promoting an agent or skill from experimental to production status.

9. Model Gateway and AI FinOps Plane

Keep the architecture portable across models and providers. Define:

- model routing by task complexity and risk;
- structured-output enforcement;
- fallback models;
- provider failure handling;
- caching;
- prompt and context budgets;
- latency budgets;
- inference-cost budgets;
- batch processing;
- rate limiting;
- model-version evaluation; and
- provider exit strategy.

The core economic measure should be cost per verified business outcome, not cost per conversation.

10. Observability and Operations Plane

Define end-to-end observability for agent runs, skill execution, tool calls, workflow state, model latency, token usage, cost, errors, retries, authorization denials, approval delays, evidence creation, action completion, user corrections, and business outcomes.

Include correlation and trace identifiers, redacted logs, dashboards, alerting, SLOs, error budgets, runbooks, incident ownership, rollback, and disaster recovery.

11. Country-Pack and Localization Plane

Design country packs as governed, versioned capabilities rather than hard-coded prompt fragments.

Country packs should support jurisdiction, effective dates, regulatory source provenance, expert reviewer, language and terminology, currency, tax and statutory configuration, accounting mappings, payment-provider patterns, business calendars, evidence rules, tests, and deprecation.

Explain how OHADA/SYSCOHADA foundations and country-specific requirements interact without contaminating the common platform core.

12. Ecosystem and Extension Plane

Design a future extension model for internal teams and approved partners. Define:

- agent and skill SDK;
- sandbox;
- manifest format;
- local testing;
- certification;
- publishing;
- permissions;
- commercial packaging;
- usage metering;
- revenue sharing;
- quality monitoring;
- suspension;
- revocation; and
- compatibility guarantees.

Do not recommend opening the ecosystem before the internal registry, tool gateway, evaluations, and governance are production-ready.

Agent Taxonomy to Evaluate

Evaluate whether the platform should include:

- personal work assistants;
- role copilots;
- domain agents;
- workflow agents;
- control and assurance agents;
- monitoring agents;
- knowledge agents;
- orchestration agents;
- evaluation agents;
- security and policy agents;
- support agents;
- integration agents; and
- platform-operations agents.

For every recommended agent, specify its business purpose, user, trigger, inputs, outputs, skills used, required tools, permission scope, human-approval point, failure behavior, evidence produced, KPIs, and implementation priority.

Skill Taxonomy to Evaluate

Evaluate reusable skills for:

- daily operating truth;
- navigation and explanation;
- inventory intelligence;
- stock variance investigation;
- replenishment;
- sales and margin intelligence;
- cash visibility;
- payment reconciliation;
- purchasing and accounts payable;
- supplier controls;
- accounting close readiness;
- compliance preparation;
- payroll input readiness;
- document and evidence management;
- fraud and anomaly controls;
- forecasting;
- customer and support operations;
- onboarding; and
- country-specific workflows.

Identify which skills should be deterministic, model-assisted, retrieval-assisted, workflow-based, human-reviewed, or prohibited from autonomous execution.

Stickiness and Value Design

Explain precisely how the architecture can create legitimate product stickiness through accumulated customer value rather than artificial lock-in.

Evaluate mechanisms including:

- role-specific daily briefs;
- persistent exception and action queues;
- workflow continuity;
- organizational operating memory;
- approved playbooks;
- personalized but governed recommendations;
- outcome history;
- multi-role collaboration;
- country-specific control packs;
- trusted evidence;

- integrations; and
- measurable business improvements.

For every proposed stickiness mechanism, define its target user, recurring trigger, user problem solved, expected frequency, measurable outcome, required data, risk, and moat potential.

Required Architecture Decisions

Explicitly decide and justify:

- 1 Central orchestrator versus federated domain orchestration.
- 2 Agent runtime versus deterministic workflow engine responsibilities.
- 3 Synchronous versus event-driven execution.
- 4 Shared platform memory versus domain-owned memory.
- 5 Agent composition versus skill composition.
- 6 Central tool gateway versus direct service access.
- 7 Model-provider abstraction strategy.
- 8 Human-approval and maker-checker architecture.
- 9 Versioning and compatibility model.
- 10 Country-pack separation.
- 11 Multi-tenant scaling model.
- 12 Build-versus-buy boundaries.
- 13 Initial deployment topology.
- 14 Disaster recovery and graceful degradation.
- 15 How the product behaves when AI services are unavailable.

For each decision, document the available options, advantages, disadvantages, risks, recommendation, rationale, and conditions that would require revisiting it.

Required Deliverables

1. Executive Architecture Report

Include an executive summary, verified current-state assessment, principal gaps, target architecture, strategic value, feasibility, risks, cost and complexity considerations, and final recommendation.

2. Architecture Diagrams

Provide diagrams for:

- system context;
- logical architecture;
- request and orchestration flow;
- event and workflow spine;
- tool authorization flow;
- knowledge and memory boundaries;
- agent and skill lifecycle;

- observability and evaluation; and
- deployment topology.

Use Mermaid where practical.

3. Agent and Skill Placement Matrix

Map every current and proposed agent and skill to its architecture layer, owning domain, users, tools, data, risk, autonomy level, dependencies, and delivery phase.

4. Canonical Contracts

Propose schemas or interfaces for:

- agent manifest;
- skill manifest;
- tool contract;
- business event;
- workflow case;
- evidence record;
- approval request;
- agent-run trace;
- evaluation result;
- country-pack manifest; and
- outcome record.

5. Phased Roadmap

Provide:

- Phase 0: discovery and governance;
- Phase 1: platform foundation;
- Phase 2: first high-value workflows;
- Phase 3: cross-module orchestration;
- Phase 4: country packs and ecosystem; and
- Phase 5: policy-bounded automation.

For every phase, identify objectives, dependencies, deliverables, responsible disciplines, risks, exit gates, and measurable customer outcomes.

6. Prioritized Backlog

Classify recommendations as:

- P0: foundational and non-negotiable;
- P1: required for the first valuable release;
- P2: scaling and differentiation; and
- P3: future ecosystem or advanced autonomy.

Include estimated complexity, dependencies, and value.

7. Risk Register

Cover security, tenant isolation, financial correctness, privacy, hallucination, incorrect tool execution, stale regulatory knowledge, model-provider dependency, operational cost, latency, observability gaps, user overreliance, poor adoption, agent duplication, and architectural fragmentation.

8. Measurement Framework

Define metrics for activation, value-active usage, workflow completion, exception-resolution time, recommendation acceptance, material correction, evidence coverage, safety, reliability, cost per outcome, retention, expansion, and customer lifetime value.

9. Implementation-Ready Follow-Up Plan

Identify the smallest architectural slice that can be implemented safely to prove the backbone without prematurely building the entire vision.

The recommended first slice should demonstrate:

- agent and skill registration;
- secure tool invocation;
- evidence-backed output;
- approval control;
- run tracing;
- evaluation; and
- one measurable Stoquify workflow.

Execution Checklist

- 1 Establish a verified inventory of the current platform, agents, skills, tools, and controls.
- 2 Identify overlaps, gaps, unsafe boundaries, and reusable foundations.
- 3 Define the architecture principles and required decisions before selecting technologies.
- 4 Design the layered target architecture and its canonical contracts.
- 5 Map every current capability into the proposed structure.
- 6 Define the first implementation slice and its release gates.
- 7 Produce the phased roadmap, backlog, risk register, and measurement framework.
- 8 Verify all repository-specific claims against code, graphs, reports, and tests.
- 9 Save the completed architecture package and supporting evidence.

Risk Controls

The proposed architecture must preserve these non-negotiable boundaries:

- No agent may infer or bypass permissions.
- No direct database writes from language-model-generated logic.
- No cross-tenant memory or retrieval.
- No authoritative financial calculations outside deterministic domain services.
- No high-impact execution without proportional approval and audit.

- No unsupported legal, tax, payroll, or accounting claims.
- No regulatory rule without source provenance and effective dates.
- No production agent or skill without evaluation, ownership, versioning, and rollback.
- No sensitive information in unredacted prompts, traces, logs, or model-provider payloads.
- No uncontrolled recursive agent delegation.
- No autonomy without a bounded policy, budget, timeout, and failure path.
- No dependency on a single model provider without an exit strategy.

Non-Goals

Do not:

- implement a generic chatbot and call it the Copilot backbone;
- replace deterministic business services with model reasoning;
- propose agents merely because they sound innovative;
- build every agent before the shared control plane exists;
- hard-code country or statutory rules into prompts;
- introduce unrelated repository refactors;
- clean up unrelated lint or type errors;
- modify existing behavior without evidence and tests; or
- claim current capabilities that the repository does not contain.

Preserve the dirty worktree and do not overwrite unrelated user changes.

Verification Requirements

Use focused verification appropriate to the inspected and implemented slice. At minimum:

- validate registry and manifest schemas;
- test authorization and tenant isolation around every exposed tool;
- test structured outputs and failure behavior;
- verify audit, evidence, and trace creation;
- verify idempotency and approval enforcement for write-capable tools;
- run relevant focused unit and integration tests;
- perform route or workflow smoke tests where applicable;
- record skipped checks and unresolved blockers honestly; and
- avoid claiming production readiness without the defined release evidence.

Success Criteria

The work is complete only when:

- 1 Every existing agent and skill has a justified place, consolidation path, or retirement recommendation.
- 2 New agents and skills can be added through stable contracts rather than bespoke integrations.

- 3 The architecture separates orchestration, business truth, tools, knowledge, policy, evaluation, and experience.
- 4 All reads and actions remain tenant-safe, role-safe, entitlement-safe, and auditable.
- 5 High-impact actions have explicit approval, evidence, idempotency, and rollback controls.
- 6 The architecture supports both current advisory use and future policy-bounded automation.
- 7 The first implementation slice is narrow, feasible, measurable, and valuable.
- 8 The roadmap contains clear dependencies and release gates.
- 9 The value and stickiness mechanisms are tied to measurable user outcomes.
- 10 The design can degrade safely when models, tools, events, or external providers fail.
- 11 The resulting system is maintainable, model-portable, cost-controlled, and capable of supporting country and partner expansion.
- 12 All conclusions are separated into verified facts, inferences, and recommendations.

Final Output

Save the completed architecture package in both Markdown and PDF formats under:

docs/copilot/

Suggested filenames:

- STOQUIFY_FUTURE_READY_COPILOT_BACKBONE_ARCHITECTURE.md
- STOQUIFY_FUTURE_READY_COPILOT_BACKBONE_ARCHITECTURE.pdf

Lead with the recommended architecture and the smallest safe implementation slice. Include command results, inspected evidence, unresolved uncertainties, and any claims that still require validation.